

UM–SJTU JOINT INSTITUTE
VE490 UNDERGRADUATE RESEARCH PROGRAM
FINAL REPORT

**High-level Synthesis:
A C-to-Verilog Approach**

AUTHOR: WENJING ZHANG
INSTRUCTOR: PROF. WEIKANG QIAN

August 1, 2023

Abstract

In this research program, we use TDHLS as the high-level synthesis tool to generate the Verilog code for the given C code. First of all, we will systematically review the basic concepts about HLS and the overall structure of TDHLS. Then, we will focus on the feature of supporting the high-level synthesis of the specific function, which is aimed at calculating the latency of the given function. This report will include the previous model used to support the feature of specific function and the improvements with two counters. Additionally, we will talk about the automated identification to specify the specific function.

Acknowledgement

First of all, I would like to thank Professor Weikang Qian for his dedicated guidance and instruction to the research group, which ensure an efficient and high-quality research process. I would also like to thank Jian Shi and Xingyue Qian, who have been contributing to the overall design of TDHLS for several years. Additionally, I would like to thank other undergraduate students, including Jiaou Chen, Yan yan, Zhijun Bian, Ting Lin, and Zining Wang, for implementing some features and collecting statistical data.

Contents

1	Introduction	4
2	Background	5
2.1	LLVM Compiler Infrastructure	5
2.2	TDHLS Structure	6
2.2.1	RTL Generation	6
2.2.2	Example: from <i>printf</i> to <i>\$display</i>	7
2.3	Module Architecture	9
3	RTL Design for Specific Function	9
3.1	Background	9
3.2	Architecture Design	10
3.2.1	Model1: Specific function as the independent top module	10
3.2.2	Model2: Counters added in the main module	11
3.3	Results and Verification	13
3.4	Automated Identification	14
4	Conclusion and Reflection	15
5	Appendix	17
5.1	Implementation of converting <i>printf</i> to <i>display</i>	17
5.2	Implementation of specific function feature	17

1 Introduction

In modern computer architecture, most Integrated Circuits (ICs) are now heterogeneous System-on-Chip (SoCs). In consideration of the significantly increasing complexity of using the low-level Hardware Description Language (HDL) to build the heterogeneous SoCs, it would be more efficient for engineers to work at a higher level of abstraction using high-level languages.^[1] High-level Synthesis (HLS) has been widely applied to this issue. HLS is known as the procedure of compiling the code written in high-level language (such as C/C++/Python) into hardware-description language (such as Verilog and VHDL), which can be described as the process of converting behavioral descriptions into efficient RTL.^[2]

The overall procedure of HLS can be divided into three stages: front-end, main synthesis, and back-end. The first stage transforms the software source code into the Intermediate Representation (IR), and implements common optimizations through the compiler before entering the next stage. The main synthesis stage takes IR as the input, executing allocation, scheduling^[3], and binding, where the Control Data Flow Graph (CDFG) and the state table are constructed. In the back-end stage, the structural RTL code in HDL is generated, based on given inputs from the previous stages.

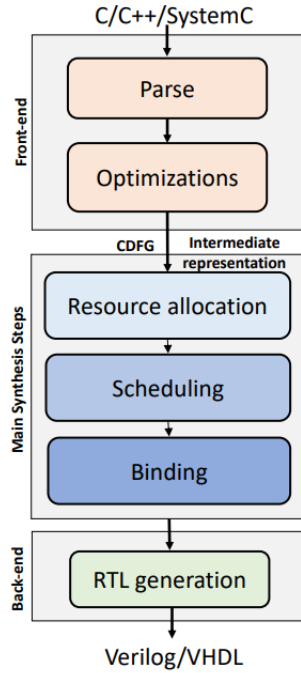


Figure 1: HLS Designing Process

In this report, we will first introduce LLVM compiler infrastructure and LLVM IR. Then, we will talk about the basic structure of our HLS tool and how the HLS design is processing through TDHLS. Then, we will mainly focus on the back-end stage regarding RTL generation in TDHLS, followed by the detailed implementation of the feature of generating RTL design for the specific function.

2 Background

2.1 LLVM Compiler Infrastructure

HLS is closely associated with compiler technology. Currently, the dominant HLS compilers are implemented based on the LLVM compiler infrastructure. The three-phase design of LLVM compiler ensures the flexibility to support multiple source languages in front-end and target architectures in the back-end, as is shown in Fig.2.

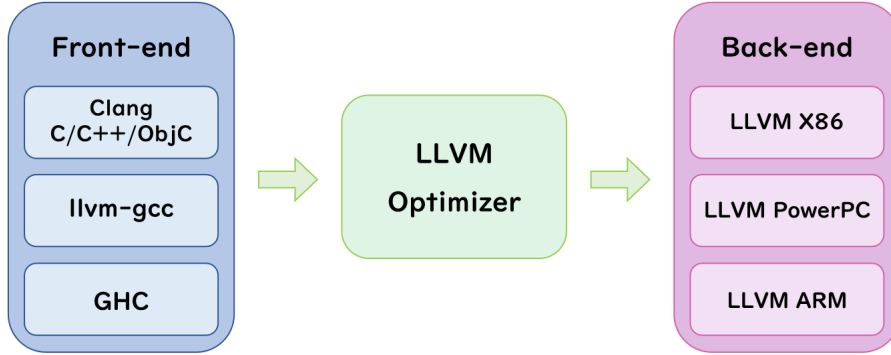


Figure 2: LLVM Three-phase Design

In spite of the inconsistency in source languages, we can still generate the target machine through the common LLVM optimizer, where the codes are represented in LLVM Intermediate Representation (IR). LLVM IR is a common and expressive form generated from the front-end, on which LLVM optimizer can perform passes. There are three isomorphic formats of LLVM IR: the textual format, the in-memory data structure format, and the bytecode format.^[4] Since the in-memory format and the bytecode format (.bc) are not legible, we will focus on the textual format (.ll) in the following discussion.

Fig.3 shows a partial IR file add.ll generated from the source file add.c. The basic block contains several instructions to acquire variables, which are represented with the prefix “%”. For specific, we use *load* instruction to acquire *tmp* from the global variables *a*. Additionally, we use *call* instructions to call the functions *dsp* and *printf*.

```
; Function Attrs: nounwind uwtable
define dso_local i32 @main() local_unnamed_addr #1 {
bb:
    %tmp = load i32, i32* @a, align 4, !tbaa !2
    %tmp1 = load i32, i32* @b, align 4, !tbaa !2
    %tmp2 = load i32, i32* @c, align 4, !tbaa !2
    %tmp3 = load i32, i32* @d, align 4, !tbaa !2
    %tmp4 = call i32 @dsp(i32 %tmp, i32 %tmp1, i32 %tmp2, i32 %tmp3) #3
    %tmp5 = call i32 (i8*, ...) @printf(i8* getelementptr inbounds ([4 x i8], [4 x i8]* @.str, i64 0, i64 0), i32 %tmp4) #4
    %tmp6 = icmp ne i32 %tmp4, 13
    %tmp7 = zext i1 %tmp6 to i32
    ret i32 %tmp7
}
```

Figure 3: add.ll

In TDHLS, we will use LLVM library to parse instructions in IR to construct the object code.

2.2 TDHLS Structure

2.2.1 RTL Generation

The overall structure of TDHLS is designed according to the HLS process, including LLVM compilation, allocation, scheduling, binding, and RTL generation.^[5] We will mainly talk about implementation of RTL generation in TDHLS, which parses IR instructions to generate the corresponding RTL design in Verilog.

To construct the RTL design, we define classes for variables, expressions, blocks and modules. The hierarchy diagrams for the RTL variables and blocks are shown below.

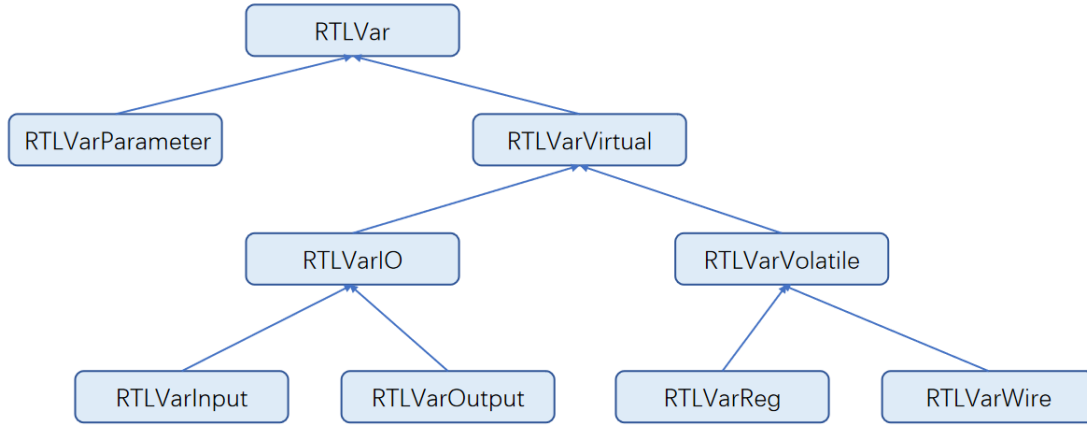


Figure 4: Hierarchy Diagrams of RTL Variables

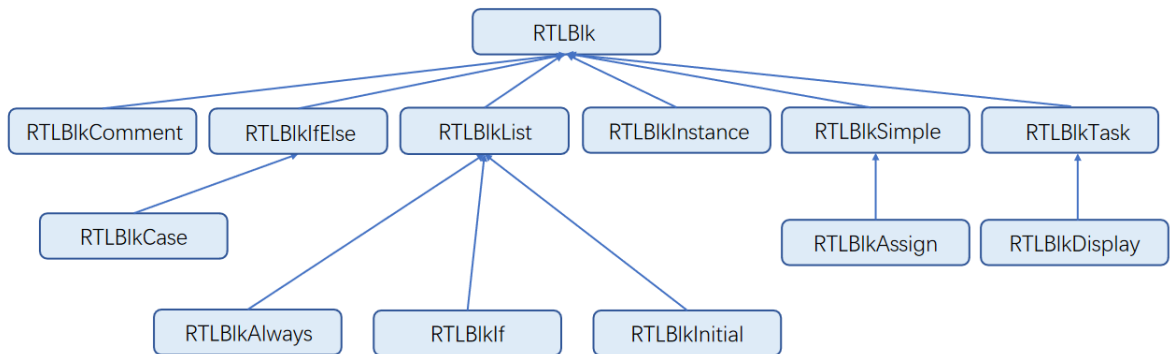


Figure 5: Hierarchy Diagrams of RTL Blocks

All the variables and blocks are defined properly according to the Verilog syntax. With this systematical hierarchy, TDHLS is able to generate all the possible always block

and initial block that includes simple non-blocking or blocking assignments, as well as the conditional statement such as *if-else* and task such as *\$display*. The following algorithm shows the simplified implementation of generating a simple always block in Verilog by using RTL generator:

Algorithm 1 Generate always block with multiple statements

```

1: Create a block list;
2: for statement i in the current block do
3:   if statement i is if-else then
4:     Create an if-else block;
5:     Create an expression of the condition statement;
6:     Create a sub block list, operate step 2 recursively;
7:     Add the condition and the sub block list to the if-else block;
8:     Add the if-else block to the block list;
9:   else if statement i is assignment then
10:    if non-blocking then
11:      Create a non-blocking assignment block;
12:    else if blocking then
13:      Create a simple assignment block;
14:    end if
15:    Add the assignment block to the block list
16:  end if
17: end for
18: if always(posedge clk) then
19:   generate Posedge shell for the block list;
20: else if always(*) then
21:   generate simple shell for the block list;
22: end if

```

2.2.2 Example: from *printf* to *\$display*

To construct the RTL design, we also define a class called RTLGenerator to generate those components based on the given instructions in IR. For example, if we want to convert the specific function *printf* in C/C++ into *\$display* in verilog, we can implement this feature through the following derived class:

```

class RTLBlkDisplay : public RTLBlkTask {
public:
    RTLBlkDisplay () { type_ = DISPLAY; }
    void AddDisplayFormat ( std::string disp_format_ );
    void AddDisplayVar ( RTLExpPtr disp_var_ );
    std::string GetStr() const;
private:
    std::vector<RTLExpPtr> disp_;
    std::string format_;
};

```

Since *\$display* is classified as a task type in Verilog, we first define a derived class called RTLBlkDisplay, inheriting the RTLBlkTask class. We use the private attribute

format_ to store the output format of *printf*, and use a vector *disp_* to store variables that are going to be displayed.

In order to acquire display format and variables, we should first check the equivalent form of *printf* in IR.

```
call i32 (i8*, ...) @printf(i8* getelementptr inbounds
    ([4 x i8], [4 x i8]* @.str, i64 0, i64 0), i32 %tmp4) #4
```

As can be seen from above, the first argument “i8* getelementptr inbounds ([4 x i8], [4 x i8]* @.str, i64 0, i64 0)” represents the display format, while the second argument “i32 %tmp4” stands for the variables to be displayed. To parse the arguments respectively, we can utilize the functions provided by LLVM library^[4]:

- `getCalledFunction()`: return the function called by the “call” instruction
- `getNumArgOperands()`: return the number of the parameters of the function
- `getOperand(int i)`: return the *i*th argument of the function
- `getArgOperand(int i)`: return the *i*th argument of the function being called
- `getInitializer()`: return the expression of the initial value of the given variable
- `getAsCString()`: return the string representing the given variable

By utilizing the above functions in LLVM library, we can easily store the first argument of the instruction as the display format in the display block, and the rest of the arguments will be stored in the vector of the variables to be displayed. The pseudocode below shows the general procedure of parsing the arguments of *@printf* in IR.

Algorithm 2 Convert *printf* into *display*

```

1: Use getCalledFunction() to parse the instruction;
2: if instruction is printf then
3:   Create a display block;
4:   for i = 0,...,getNumArgOperands()-1 do
5:     Obtain the ith argument by using getArgOperand(i);
6:     if i = 0 then
7:       Convert the ith argument to from Value type to string;
8:       Store the obtained string as the display format in the display block;
9:     else
10:      Create an RTL variable with value of the ith argument;
11:      Store the created variable as the display variable in the display block;
12:    end if
13:  end for
14:  Create an if-else block;
15:  Create an expression of the display condition;
16:  Add the display block and condition to the if-else block;
17:  Generate shell for the if-else block;
18: end if
```

2.3 Module Architecture

After completing RTL generation, we will obtain a series of Verilog files (.v) describing the RTL modules generated by TDHLS. The dominant architecture is composed of the following four modules:

- top module: invokes the main module and the memory controller
- main module: describes the main function, invoking the sub-module if it exists
- memory controller: manages and schedules memory access, controlling ram ports
- ram: basic block storage unit, including dual port ram and dual port struct ram

Each of the module is described by the Verilog file, including top.v, main.v, mem_controller.v, and ram.v. In order to test the correctness of our RTL design, we also create a testbench.v file that performs the simulation on the top module. If the return value obtained at the end of the simulation is 0, we could regard it as the passed test, otherwise, it is regarded as a failed test. The following Fig.6 illustrates the module architecture of our RTL design.

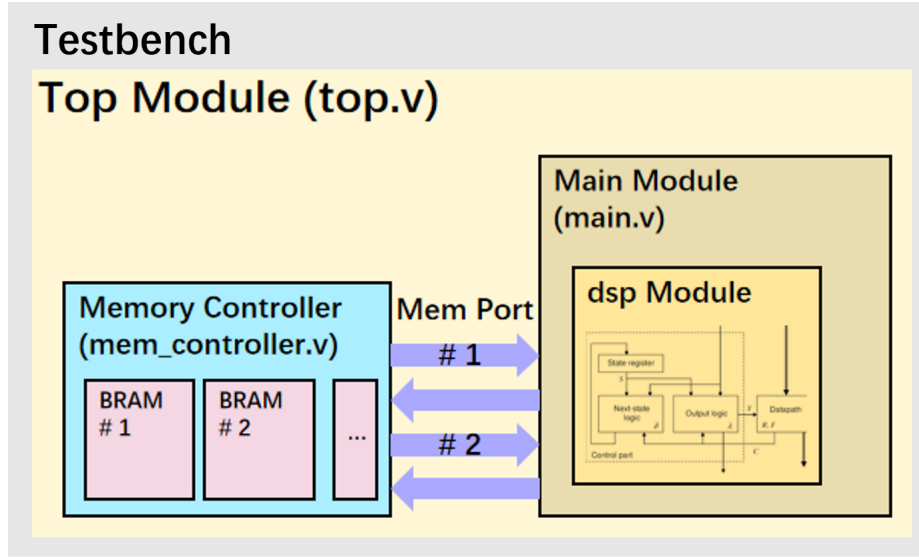


Figure 6: Module Architecture of the RTL Design

3 RTL Design for Specific Function

3.1 Background

Based on the current implementation, TDHLS is able to generate the RTL design for the entire program. In order to analyze the performances of TDHLS comparing to Xilinx Vivado HLS, we have tested multiple cases and recorded the latency under TDHLS and Vivado HLS respectively. The results are shown in Tab.1.

Table 1: Latency for each case testing under Vivado HLS and TDHLS

Example	Latency (clock cycles)		Relative Difference (%)
	Vivado HLS	TDHLS	
adpcm	37742	57252	51.9
aes	5550	12531	125.78
bf	160609	362357	125.61
dfadd	7	4073	58085.71
dfdiv	91	2447	2589.01
dfmul	8	1551	19287.50
dfsin	3500	116458	3227.37
gsm	5204	11485	120.70
mips	4294	11344	164.18
sha	201935	338194	67.48

Calculating the relative difference for each case under two HLS tools, we can obviously see that all the cases testing under TDHLS have higher latency than those under Vivado HLS. In spite of the relatively smaller ratio for *adpcm* and *sha*, we still observe several cases with large discrepancy that exceeds 1000%, by which *dfadd*, *dfdiv*, *dfmul*, and *dfsin* characterized.

Through further investigation, we have found that the latency result in Vivado HLS is calculated by focusing on the specific function for a single call. In contrast, TDHLS tests the total latency of the entire program, including the main function and the specific functions, resulting in a large difference when the specific function is called for multiple times. To resolve this issue, it is of high priority to generate RTL design for the specific function instead of the entire program.

3.2 Architecture Design

3.2.1 Model1: Specific function as the independent top module

Considering that the previous architecture in Fig.6 embeds the module of the specific function inside the main module, it would be hard for us to obtain the RTL design of the function directly. Therefore, we have designed the following model to implement the feature of specific function.

As is shown in Fig.7, we separate the main module and the module of the specific function. The top module is now used for testing the specific function, while the main top module is functioned as the top module in previous architecture. Moreover, the additional memory controller is connected to the module of the specific function, and the original one is remained connected to the main module.

Despite the ideal construction on this architecture, we fail to test some of the cases that modify the memory controller in the specific function. Generally speaking, the information stored in the two memory controllers are not synchronous, leading to testing failure in several cases.

In order to solve this problem, we can delete one of the memory controllers. However, this would derive another issue regarding using a large mux^[6] to decide whether to connect the main module or the module of the specific function, which not only increases the complexity of the circuit, but also brings about some potential problems.

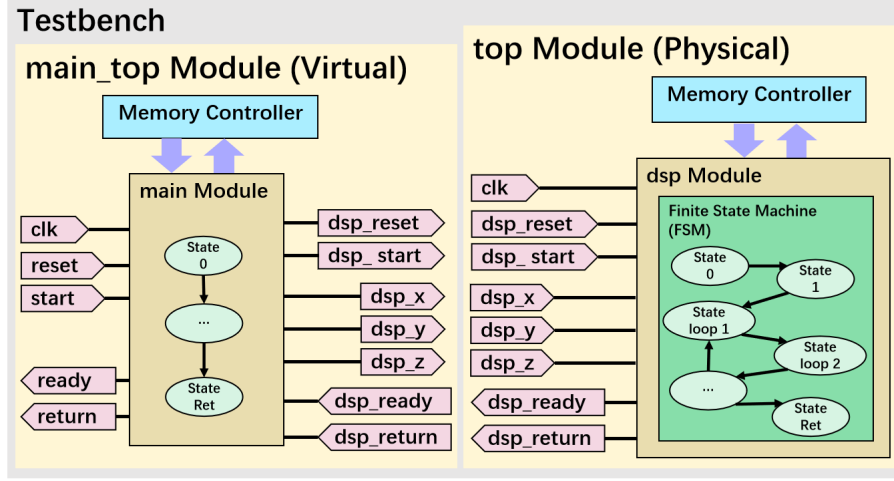


Figure 7: Module Architecture of the Specific Function

3.2.2 Model2: Counters added in the main module

Given that the implementation in Model1 is more complicated, we designed another model based on the original architecture.

Recall that the main goal we would like to achieved is calculating the latency of the specific function for a single call. It is also feasible to use counters to complete the task briefly. In the revised architecture, we have designed two counters in the main module to calculate the latency according to start, ready and reset signals:

- `count_call`: counts the total times that the function is called by the main function
- `count_clk`: records the total latency that the function is called by the main function
- `func_count_clk`: counts the latency that the function is called by the main function
- `reset`: the reset signal of the main function
- `start`: the start signal of the main function
- `func_start`: the start signal of the specific function
- `func_ready`: the ready signal of the specific function

When the `reset` signal equals 0, both `count_call` and `count_clk` are set to 0. To calculate `count_call`, we only need to increase it by 1 when `func_ready` signal is on the rising edge. As for `count_clk`, we indirectly use `func_count_clk` to count the latency for each call on the specific function, which avoids the possible delay. When `func_start` signal equals 1, `func_count_clk` increases by 1 for each clock cycle, until `func_ready` signal becomes 1. Subsequently, `count_clk` is added with `func_count_clk`, which records the latency of the current call. Then, `func_count_clk` is set to 0 until the next call begins.

For example, we would like to test the latency of `dsp` function in `add` example, the architecture of which is shown in Fig.8.

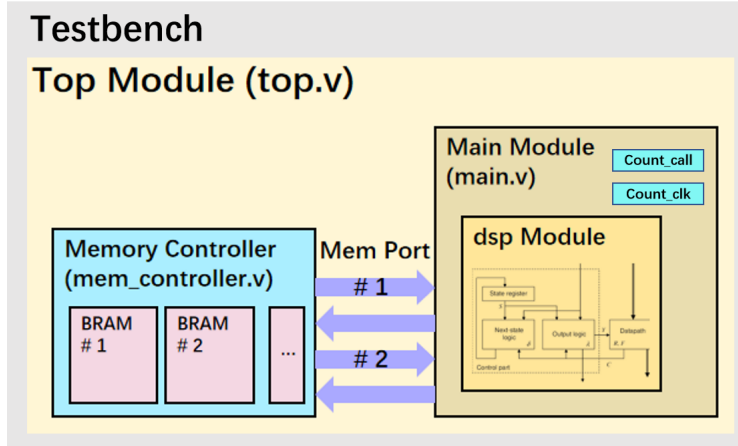


Figure 8: Revised Module Architecture of the Specific Function

Based on this architecture, we use RTL generator in TDHLS to generate the following Verilog code with counting signals and four basic blocks:

```

always @( posedge clk ) begin
    if ( reset == 1'b1 ) begin
        dsp_reset <= 1'b1;
        dsp_start <= 1'b0;
        func_count_clk <= 1'b0;
    end else if ( start == 1'b1 ) begin
        dsp_reset <= dsp_reset_next;
        dsp_start <= dsp_start_next;
        func_count_clk <= func_count_clk_nxt;
    end
end

always @( posedge clk ) begin
    if ( reset == 1'b1 ) begin
        count_clk <= 32'b0;
        count_call <= 32'b0;
    end
end

always @( * ) begin
    if ( dsp_start == 1'b1 ) begin
        func_count_clk_nxt <= func_count_clk + 1;
    end else begin
        func_count_clk_nxt <= 32'b0;
    end
end

always @( posedge dsp_ready ) begin
    count_call <= count_call + 1;
    count_clk <= count_clk + func_count_clk;
end

```

```

integer f1;
always @(posedge ready) begin
    f1 = $fopen("count.txt", "w");
    #1 $fwrite(f1, "clk cycle count = %1d\nfunction call count
               = %1d\naverage clk count = %1d\n", count_clk,
               count_call, count_clk/count_call);
    #1 $fclose(f1);
    #754 $display("$display excuting result: %d.", count_clk);
end

```

After obtaining the value of the two counters, we divide *count_clk* by *count_call* to acquire the average latency for a single call. By dumping the results into *count.txt*, we can obtain the information including the total latency, the times of the function call, and the average latency, as presented in Fig.9.

```

examples > add > ≡ count.txt
1    clk cycle count = 5
2    function call count = 1
3    average clk count = 5

```

Figure 9: count.txt of add

3.3 Results and Verification

The revised architecture with counters implemented is feasible to count the latency of a single function call. In order to test the correctness of the feature, we can perform the behavioral simulation in iverilog. Through the observation on the variation of *count_clk* and *count_call* according to different signals, we can tell whether the counting results are consistent with the actual latency. The simulation result for *add* example is illustrated in Fig.10.

Comparing the results in Fig.9 and Fig.10, we found that the latency of *dsp* function is exactly 5 clock cycles with one single call. Through further verification by following similar procedure on other cases, the feature of specific function is proved correct.



Figure 10: Simulation result for add example

Utilizing the new feature to count the latency, we can test the previous cases again to see if the difference of the results between Vivado HLS and TDHLS becomes smaller. The specific details are shown in Tab.2.

Table 2: Latency for each case testing under Vivado HLS and TDHLS

Example	Latency (clock cycles)		count call	Relative Difference (%)
	Vivado HLS	TDHLS		
adpcm	37742	57634	1	52.71
aes	5550	13399	1	141.42
bf	160609	362353	1	125.61
dfadd	7	78	46	1014.29
dfdiv	91	102	22	12.09
dfmul	8	68	20	750.00
dfsin	3500	3264	36	-6.74
gsm	5204	10061	1	93.33
mips	4294	12059	1	180.83
sha	201935	352903	1	74.76

As can be seen from the table above, there exists a significant reduction in the disparity of *dfadd*, *dfdiv*, *dfmul*, and *dfsin*, comparing to the previous results in Tab.1. Since those four cases include multiple calls on the specific function, the previous implementation that calculates the latency for the whole program inevitably results to the large disparity. After applying the new features with counters, the average latency calculated by TDHLS is closer to the results performed by Vivado HLS.

3.4 Automated Identification

In order to generate the previous Verilog code to count the latency, we declare several counting signals and create four *always* blocks in the main module by using RTL generator of TDHLS. It traverses all the functions to find the target one that is going to be tested. Given that there might be multiple specific functions called by the main function, we would like to specify the function name in Makefile, and design the automated identification to parse the target function name in TDHLS. The overall procedure can be simplified through Fig.11 below.



Figure 11: Automated Identification of the Specific Function

First of all, we add an option called “FUNCTION” in Makefile, of which the parameter can be specified as the target function name according to the user’s needs. For example, if we are going to test the latency of *float64_div* function in *dfdiv* case, we can add the following option in Makefile:

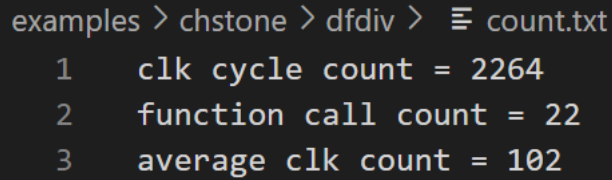
```
FUNCTION = float64_div
```

This option indicates that the name of the target function is “float64_div”. Correspondingly, we create a new option type in Target Machine that allowed TDHLS to parse this function name argument.

```
static cl::opt<std::string>
    FunctionName("func", cl::desc("Target Function in HLS"),
        cl::value_desc("funcname"));
```

The code above defines a static command-line option named “FunctionName” using the `cl::opt` class from the LLVM library. It enables TDHLS to read the command with the format “-func=\$(FUNCTION)”. For example, when the makefile of *dfdiv* is added with “FUNCTION = float64_div”, then it will read “-func=float64_div” in the command line and store *float64_div* as a string in the program.

At first, the string of the target function name is stored in Target Machine, then it will be propagated to HLS Pass, and finally become part of the attributes in RTL generator. As is mentioned in the previous section, RTL generator will decide signals of which function to be tracked and activate the counters according to the current values of its *start* and *ready* signals. For example, both *float64_div_start* and *float64_div_ready* will be taken into consideration when testing the latency of *float64_div* function in *dfdiv* case. After a series of operations, the detailed information of the call time and latency will be dumped into *count.txt*, which is shown in Fig.12.



```
examples > chstone > dfdiv > ≡ count.txt
1    clk cycle count = 2264
2    function call count = 22
3    average clk count = 102
```

Figure 12: *count.txt* of *dfdiv*

The implementation of automated identification greatly increases the efficiency of testing the latency of the specific function. Moreover, it provides a flexible option to allow latency test on different functions that are called by the main function based on the user’s needs. After modifying the Makefile of all the previous tested cases, we obtain exactly the same results as before, which also demonstrates the feasibility of the automated identification of the specific function.

4 Conclusion and Reflection

Our process in developing the features of TDHLS is steady and efficient. Starting from the simple example about converting *printf* in C/C++ into *display* in Verilog, we have gained a better understanding of the basic structure of TDHLS and how the RTL generator works. Comparing the performances between Vivado HLS and TDHLS, we have found the discrepancy in latency and decided to implement the feature of specific function. Despite the failure in our first model with two memory controllers, we still design another architecture that is feasible to complete the latency test of a single function call. As a result, we successfully reduce the ratio of difference between the results tested under two HLS tools. Moreover, we improve the flexibility and practicality by utilizing the automated identification, which enables the user to specify the function according to their demands. The future work will focus on the completion of the documentation, including the file description and the user instructions. We will also be required to merge different feature branches together and complete the final stage of the research project.

References

- [1] B. Carrion Schaefer. High-level synthesis made easy. 2023. <https://www.hlsbook.com/>. 1
- [2] Prof. Weikang Qian. Proposal for ve/vm/mse490 undergraduate research course. 2022. <https://jicanvas.com/courses/124/files/108481?wrap=1>. 1
- [3] Lu Xiao, Zheng Xiao, Fan Wu, Yunchuan Qin, and Kenli Li. Optimization on operation sorting for hls scheduling algorithms. *Integration*, 93:102058, 2023. 1
- [4] Chris Lattner. The architecture of open source applications. <https://aosabook.org/en/v1/11vm.html>. 2.1, 2.2.2
- [5] Coussy Philippe, Meredith Michael, Gajski Daniel D., and Takach Andres. An introduction to high-level synthesis. *IEEE Design Test of Computers*, 26:8–17, 2009. 2.2.1
- [6] Fingeroff Mike and Thomas Bollaert. High-level synthesis blue book. 2010. 3.2.1

5 Appendix

5.1 Implementation of converting *printf* to *display*

```
case llvm::Instruction::Call: {
    auto call_inst = (llvm::CallInst *)inst;
    auto called_f = call_inst->getCalledFunction();
    if (IsPrintf(called_f)){
        BlkDisplayPtr display_blk = CreateRTLDisplay();
        unsigned int argnum = call_inst->getNumArgOperands();
        for (unsigned int i = 0; i < argnum; i++){
            Value* disp_arg_val = call_inst->getArgOperand(i);
            if(!i){
                auto dispExpr = dyn_cast<ConstantExpr>(disp_arg_val);
                auto disp_format_1 = dyn_cast<GlobalVariable>
                    (dispExpr->getOperand(0));
                auto disp_format_2 = dyn_cast<ConstantDataArray>
                    (disp_format_1->getInitializer());
                display_blk->
                    AddDisplayFormat(disp_format_2->getAsCString());
            }
            else{
                std::string disp_arg_str =
                    getRTLPtrFromValue(disp_arg_val)->GetStr();
                RTLEXPtr disp_var_ptr =
                    CreateRTLEXP(std::move(disp_arg_str));
                display_blk->AddDisplayVar(std::move(disp_var_ptr));
            }
        }
        auto disp_blk = CreateRTLIfElse();
        auto disp_cond =
            CreateRTLEXP(current_state,
                        mapState2Parameter[this_state]);
        disp_blk->AddCondition(std::move(disp_cond),
                            std::move(display_blk));
        generateShell(std::move(disp_blk));
        break;
    }
}
```

5.2 Implementation of specific function feature

```
void RTLGenerator::GenerateCountSignal() {
    /* initialize count signals */
    if(rtl_module->GetName() != "main") return ;
    auto cnt_datapath = CreateRTLIfElse();
    RTLEXPtr reset_cond = CreateRTLEXP(reset, const_1b1);
    BlkList reset_cnt;
```

```

reset_cnt.emplace_back(
    CreateRTLNonBlock(count_clk, const_32b0));
reset_cnt.emplace_back(
    CreateRTLNonBlock(count_call, const_32b0));
cnt_datapath->AddCondition(
    std::move(reset_cond), std::move(reset_cnt));
generatePosedgeShell(std::move(cnt_datapath));

HlsFunctionInfoMap &hls_function_info =
    HlsFunctionInfoMap::GetInstance();
for (const auto &f_call_sig : mapFunc2CallSig) {
    Function *f = f_call_sig.first;
    if (f->isIntrinsic())
        continue;
    RTLModulePtr module =
        hls_function_info.getRTLModuleFromFunction(f);
    if (module == nullptr)
        continue;
    if (module->GetName() != specific_func)
        continue;
    CallSigPtr call_sig = f_call_sig.second;

    /* assign count_ready when call the function */
    BlkList ready_assign;
    ready_assign.emplace_back(
        CreateRTLNonBlock(count_call, count_call_nxt));
    ready_assign.emplace_back(
        CreateRTLNonBlock(count_clk, count_clk_nxt));
    BlkAlwaysPtr always_blk =
        CreateRTLAlways(std::move(ready_assign));
    always_blk->SetPosVar(call_sig->ready->GetStr());
    rtl_module->AddBlk(std::move(always_blk));

    /* Add func_count_clk when running the function */
    auto cnt_clk_datapath = CreateRTLIfElse();
    RTLExpPtr cnt_clk_cond =
        CreateRTLExp(call_sig->start_, const_1b1);
    BlkSimplePtr cnt_clk_assign =
        CreateRTLNonBlock(func_count_clk_nxt,
            fcount_clk_nxt);
    cnt_clk_datapath->AddCondition(
        std::move(cnt_clk_cond), std::move(cnt_clk_assign));
    cnt_clk_datapath->AddDefaultBlocks(
        CreateRTLNonBlock(func_count_clk_nxt, const_32b0));
    generateShell(std::move(cnt_clk_datapath));
    break;
}
}

```